

Term Project – Proposal

VPN Access Manager

Dozent: Stephan Bökelmann · sboekermann@ep1.rub.de · Technische Hochschule Georg Agricola, Bochum

Author: Rayen Ben Haj Rhouma

Module: Introduction to Database Management Systems · THGA Bochum

Submitted to: Stephan Bökelmann · sboekermann@ep1.rub.de

Planned effort: approximately 40 hours over no more than two months

1 Application domain and motivation

Companies that let employees work remotely run one or more VPN gateways so that staff can reach internal systems over an encrypted tunnel. The IT team needs to know who is connected right now, to which gateway, from which client IP, and how much traffic each session used. This matters for capacity planning (a gateway has a limited number of concurrent slots) and for basic auditing (which user connected when). Doing this by hand or in a spreadsheet is unreliable: a spreadsheet cannot guarantee that a user or gateway actually exists, cannot stop the same session being closed twice, and cannot answer “how many users are active on each gateway” without manual counting.

The proposed application is a small *VPN Access Manager*. It replaces the spreadsheet with a relational database, a REST API, and a desktop frontend. It manages **departments**, **VPN users**, **gateways**, and the **connection sessions** that link a user to a gateway. Each session records its status, the client IP, the connect and disconnect times, and the number of bytes transferred.

Usage scenario 1. An administrator opens the frontend and starts a new session for a user on the Frankfurt gateway. The system refuses the connection if that gateway has already reached its maximum number of concurrent sessions.

Usage scenario 2. A capacity manager wants an overview. The main screen lists every gateway with its region, the number of currently active sessions, and its maximum, so that gateways close to their limit can be seen at a glance.

2 Data model – ER sketch

Figure 1 shows four entity types. A **department** groups employees. A **vpn_user** is a person allowed to use the VPN and belongs to one department. A **gateway** is a VPN server in a given region with a maximum number of concurrent sessions. A **session** is one connection of a user to a gateway and stores the connection-specific data.

A department *has* many users (1:N) and a user *opens* many sessions (1:N). The central relationship

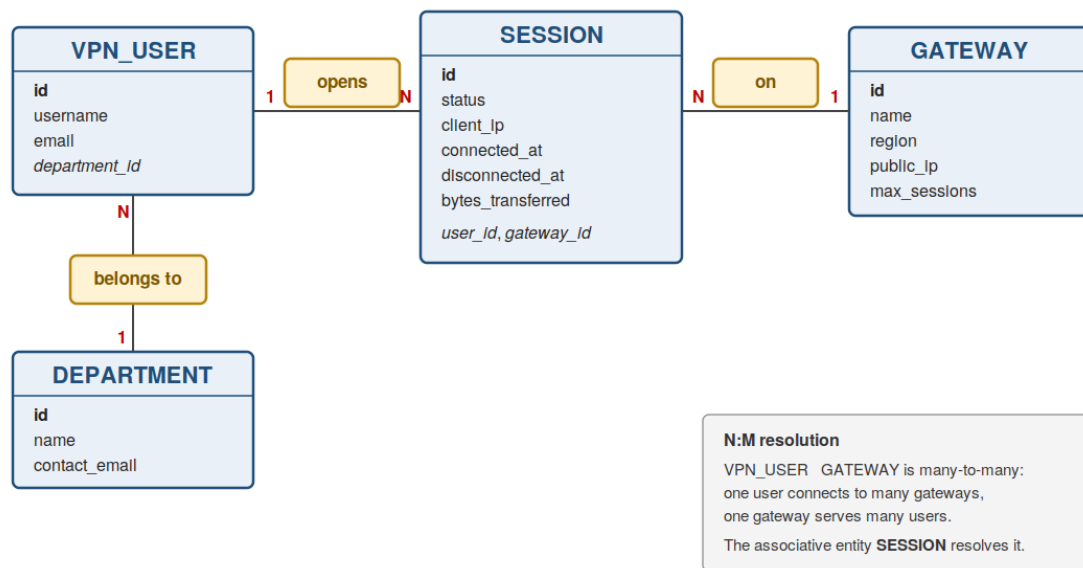


Abbildung 1: ER model of the VPN Access Manager: entity types, key attributes, foreign keys, relationships and cardinalities. The N:M relationship between `vpn_user` and `gateway` is resolved by the associative entity `session`.

is between `vpn_user` and `gateway`: one user can connect to many gateways over time, and one gateway serves many users, so `vpn_user` and `gateway` form an N:M relationship. The associative entity `session` resolves this N:M relationship into two 1:N relationships and holds the data specific to a single connection (status, client IP, times, bytes).

3 From model to schema

The relational schema is derived directly from the ER diagram. Primary keys are underlined and foreign keys are written in italics.

- `department`(id, name, contact_email)
- `vpn_user`(id, username, email, *department_id*)
- `gateway`(id, name, region, public_ip, max_sessions)
- `session`(id, status, client_ip, connected_at, disconnected_at, bytes_transferred, *user_id*, *gateway_id*)

The database uses the following integrity constraints:

- `vpn_user.username` and `gateway.name` are unique.
- `status` is restricted to `active` or `closed` by a CHECK constraint.
- `disconnected_at` must be NULL while the session is `active` and must not be earlier than `connected_at`.
- `user_id` and `gateway_id` are foreign keys with referential integrity.

- A gateway must not hold more **active** sessions than its **max_sessions** (enforced by the API before insert).

Normalisation. The schema is in third normal form (3NF). Each table describes one subject, every non-key attribute depends only on its table's primary key, and no non-key attribute depends transitively on another. Department data is stored once in **department** and referenced through **department_id**; gateway data is stored once in **gateway** and referenced through **gateway_id**; **session** stores only the data specific to a single connection.

4 API design

The REST API is implemented with FastAPI. Read operations require no API key. Write operations require a valid **X-API-Key** header. Request bodies are validated with Pydantic models, and SQL queries use parameters rather than string concatenation.

Method	Path	Purpose
GET	<code>/gateways</code>	List gateways with region and a count of active sessions (JOIN + aggregation)
GET	<code>/sessions</code>	List sessions with user, department, gateway, status, times (multiple JOINS); optional filters
GET	<code>/sessions/{id}</code>	Return the full details of one session
POST	<code>/sessions</code>	Open a session after validating FKs and the gateway capacity limit (X-API-Key)
POST	<code>/sessions/{id}/close</code>	Set an active session to closed and record the disconnect time (X-API-Key)

For example, GET `/sessions?status=active&gateway=gw-eu-1` returns only the active sessions on one gateway. The endpoint GET `/gateways` joins **gateway** and **session**, filters for active sessions, and groups by gateway to compute the number of active sessions per gateway.

5 Frontend prototype (hand sketches)

The frontend is a Python/tkinter desktop application. It talks only to the REST API and never connects to PostgreSQL directly. At startup a connection dialog asks for the API URL and the **X-API-Key**; the key is kept in memory for the session and never written to disk. The main window offers a gateway overview plus access to the session list, the open-session form, and the close action.

Figures 2–5 show the four screens of the prototype. Figure 2 shows the connection dialog that collects the API URL and **X-API-Key** at startup; Figure 3 shows the gateway overview populated by GET `/gateways`; Figure 4 shows the form that opens a new session via POST `/sessions`; and Figure 5 shows the confirmation step that closes a session via POST `/sessions/{id}/close`.

6 Operation with Docker Compose

The backend consists of two containers orchestrated with Docker Compose. The **postgres** service creates the schema and seed data from an `init.sql` script on first start. The **api** service is built from the `api/` directory, waits for a healthy database via a health check, and exposes FastAPI on port 8000.

Connect to VPN Manager

API URL
http://localhost:8000

X-API-Key

Cancel Connect

Abbildung 2: Startup connection dialog: API URL + X-API-Key.

VPN Manager — Gateways

Gateways Sessions New Session

Gateway	Region	Active	Max
gw-eu-1	Frankfurt	47	100
gw-us-1	Virginia	88	100
gw-ap-1	Singapore	12	50

GET /gateways (JOIN + COUNT active sessions per gateway)

Abbildung 3: Main screen: gateway overview, calls `GET /gateways`.

Open New Session

User r.benhajrhouma [select]

Gateway gw-eu-1 (Frankfurt) [select]

Client IP 10.8.0.14

Connected at 2026-07-18 14:22

Cancel Connect

Abbildung 4: Open-session form: calls `POST /sessions`.

Disconnect Session

User: r.benhajrhouma
Gateway: gw-eu-1 (Frankfurt)
Client IP: 10.8.0.14
Status: **active**
connected 2026-07-18 14:22

Disconnect at 2026-07-18 16:05

Cancel Disconnect

Abbildung 5: Disconnect confirmation: calls `POST /sessions/{id}/close`.

Database credentials and the API key are read from a `.env` file that is excluded from Git; a committed `.env.example` documents the required variables with placeholder values. Deployment on the lecture server is: (1) copy `.env.example` to `.env` and set secure values; (2) run `docker compose up -d --build`; (3) verify with `docker compose ps` and the API health endpoint; (4) install the frontend with `sudo dpkg -i vpn-manager_0.1.0_amd64.deb`; (5) launch `vpn-manager` and enter the API URL and key. The frontend is bundled with PyInstaller and packaged as a `.deb` with `fpm`.

7 Scope, milestones and risks

Scope. The minimum viable product contains four tables, five REST endpoints, one tkinter desktop application with four screens, Docker Compose deployment, automated tests, documentation, and a short demonstration video. Real VPN authentication, live traffic capture, per-user roles, and historical reporting dashboards are explicitly outside the scope.

Milestones and schedule.

Week	Milestone	Est. hours
1	Final ER model, SQL schema, constraints, seed data, local Docker Compose setup	8
2–3	FastAPI models and all endpoints, X-API-Key guard, JOIN and aggregation queries	12
4–5	tkinter frontend, API integration, validation, Debian packaging	12
6	Automated tests, deployment verification, documentation, video	8

Risks and mitigation.

- **Capacity-rule risk:** the “do not exceed `max_sessions`” check is the one non-trivial business rule; it is implemented in the API before insert and covered by a dedicated unit test early.
- **Packaging risk:** PyInstaller/fpm may miss dependencies; packaging is tested in a clean Debian environment, not left to the end.
- **Deployment risk:** the API may start before PostgreSQL is ready; Docker health checks and service dependencies handle this.

8 Acceptance criteria and testing

Acceptance criteria.

- The database starts from a clean Docker volume and creates all tables, constraints, and seed records without manual SQL.
- Opening a session for a non-existent user or gateway is rejected with a documented 4xx response.
- Opening a session on a gateway that is already at `max_sessions` is rejected.
- `GET /gateways` returns the correct active-session count for every gateway, including gateways with zero active sessions (JOIN + aggregation).
- `GET /sessions` correctly applies the `status` and `gateway` filters.
- Write endpoints return HTTP 401 when the API key is missing or invalid.
- Closing a session sets it to `closed` and records the disconnect time, and a second close attempt does not change it again.

Test plan.

- **Database:** verify PK/FK/UNIQUE/CHECK constraints and the time-ordering rule against a disposable test database.
- **API:** use `pytest` and FastAPI’s `TestClient` for happy paths and error cases on all five endpoints.
- **Business rule:** a test that fills a gateway to its `max_sessions` and asserts that one more open request returns a 4xx — this covers the capacity-rule risk named above.

Effort budget: the whole term project (system, documentation and video) is designed to fit into roughly **40 h of work** over at most **2 months**.